Maintenance de l'application Open WebUI Android

Équipe projet openwebui-android

Guide de maintenance, de portage et de mise à jour de l'APK **Open WebUI Android** : application autonome embarquant le serveur Open WebUI (Python), SearXNG et une base vectorielle locale, affichés dans une WebView. Ce document décrit l'architecture, les actions de portage, les ajouts par rapport au code amont, les patchs appliqués, les procédures de mise à jour d'Open WebUI et des composants, ainsi que le dépannage et la règle d'isolation de l'environnement de build.

7/11/2026

Sommaire

1. Objet du document 3
2. Architecture de l'application 4
3. Dépôt de développement 6
4. Portage Android — actions effectuées 7
   - 4.1. Phase A — Sources et cadrage 7
   - 4.2. Phase B — Patchs du code amont 7
   - 4.3. Phase C — Chaîne Android 7
   - 4.4. Phase D — Cross-build des wheels natifs 7
   - 4.5. Phase E — Outils Android x86_64 8
   - 4.6. Phase F — Pièges spécifiques 8
5. Ajouts par rapport à Open WebUI amont 10
6. Patchs du code amont 11
   - 6.1. Performances de démarrage 12
7. Mise à jour d'Open WebUI 14
8. Mise à jour des composants 15
9. Construire l'APK 16
10. Configuration sur l'appareil 17
11. Dépannage rapide 18
12. Règle d'isolation de l'environnement 20
13. Annexes 21
    - 13.1. Versions verrouillées à la livraison initiale 21
    - 13.2. Commandes utiles 21
    - 13.3. Règles de test hôte (serveur de développement) 21
    - 13.4. Intégration continue (GitHub Actions) 21
    - 13.5. Pistes d'évolution 22

1. Objet du document

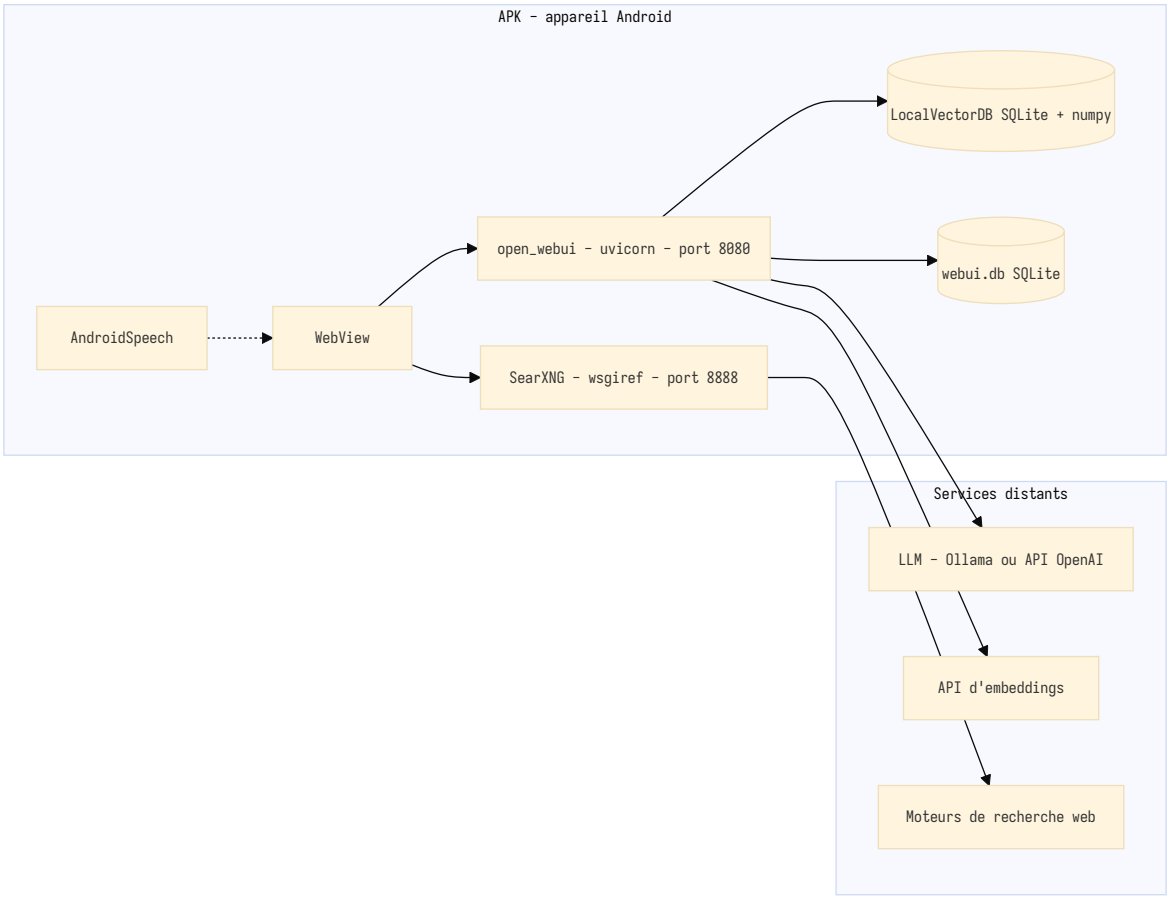
Ce document s'adresse à la personne chargée de **maintenir** l'application Open WebUI Android après la livraison initiale. Il couvre trois besoins :

1. **Comprendre** ce que contient l'APK et pourquoi le code diffère d'Open WebUI amont ;
2. **Mettre à jour** Open WebUI lui-même ou l'un de ses composants (Python, wheels natifs, SearXNG, chaîne de build) ;
3. **Diagnostiquer** les pannes connues sans refaire l'analyse d'origine.

> Toute intervention doit être tracée dans `journal.md` (horodatage UTC, décisions numérotées D1-D15+) et les patchs amont documentés dans `PATCHES.md` .

2. Architecture de l'application

L'APK est une application Android native minimale (Java) qui héberge deux serveurs Python dans un interpréteur **Chaquopy**, et affiche le tout dans une WebView pointant sur `http://127.0.0.1:8080`.



Couche	Contenu
Application Java	`MainActivity` (WebView, permissions, purge cache), `ServerService` (notification avant-plan), `AndroidSpeech` (reconnaissance vocale native)
Runtime Python	Chaquopy : CPython 3.13 embarqué, ABI `arm64-v8a`, `minSdk 24`
Backend	Open WebUI 0.13.0 **patché** (FastAPI + uvicorn + SQLite) dans `src/open_webui/`
Frontend	UI SvelteKit compilée, livrée dans le wheel amont (`open_webui/frontend/`)
Recherche web	SearXNG figé sur un commit 2026-08-28 (`src/searx/`), instance locale port 8888
RAG	Embeddings **par API** (obligatoire) + base vectorielle **locale** `LocalVectorDB`
Wheels natifs	7 wheels cross-buildés dans `wheels/` (cf. Chapitre 4.4)

Données	Stockage privé Android : `files/data/` (base SQLite, uploads, vecteurs)

3. Dépôt de développement

Tout le développement est **auto-portant** : SDK Android, Gradle, Rust, NDK, JDK, qemu et Python vivent dans le dépôt (règle d'isolation, Chapitre 12).

Chemin	Rôle
`AGENTS.md`	Objectif, périmètre, décisions, règle d'isolation
`journal.md`	Historique horodaté des actions et décisions (D1-D15)
`PATCHES.md`	Catalogue des modifications du code Open WebUI amont
`todo.md`	Suivi des tâches par phase
`requirements-min.txt`	Dépendances Python embarquées, avec justifications de pin
`vendor/unpacked/`	Wheel Open WebUI **pristine** (source des patchs)
`vendor/searxng/`	Dépôt SearXNG figé sur un commit précis
`src/`	Sources embarquées : `open_webui/` patché, `searx/`, shims `tiktoken` / `pycrdt`
`scripts/prepare_src.py`	Générateur de `src/` : copie pristine + patchs idempotents
`scripts/build-wheels.sh`	Cross-build Rust des wheels natifs Android
`scripts/build.sh`	Build complet de l'APK (SDK, wheels, Gradle)
`scripts/smoke.sh`	Test de fumée hôte : démarrage réel du serveur
`scripts/env.sh`	Ancrage d'isolation (variables d'environnement)
`app/`	Module Android (Java, manifeste, Chaquopy, ressources)
`wheels/`	Wheels Android produits (`*_android_24_arm64_v8a.whl`)
`dist/`	APK livrés (`app-debug.apk`)
`docs/`	Ce document (`maintenance.typ` , `maintenance.pdf`)

4. Portage Android — actions effectuées

Cette section est le **mémoire du portage**. Chaque item est un problème réel rencontré et sa solution, dans l'ordre chronologique.

4.1. Phase A — Sources et cadrage

- Récupération des sources Open WebUI 0.13.0 **via le wheel PyPI** (le wheel contient le frontend déjà compilé : aucun build npm nécessaire). Piège : le wheel porte `Requires-Python <3.13` — téléchargé avec un Python 3.12 portable.
- Cible **Python 3.13** (les wheels Android du dépôt Chaquopy existent en cp313). Le blocage `<3.13` amont vient de `pydub` / `audioop` (retiré de la stdlib en 3.13) : neutralisé par le patch P5.
- SearXNG figé sur le commit `a30b2d4749` (2026-08-28) : dernier état **avant** la migration vers `curl_cffi` (natif inembarquable).

4.2. Phase B — Patchs du code amont

- Développement de `scripts/prepare_src.py` : chaque modification du code amont est un **motif texte** remplaçable, vérifié, rejouable et explicitement documenté (P1-P11, cf. Chapitre 6). Un patch dont le motif a disparu **échoue fort** (pas de modification silencieuse).
- Shims embarqués : `tiktoken` (comptage approximatif), `pycrdt` (collab dégradée), `NullVectorDB`, `LocalVectorDB`.

4.3. Phase C — Chaîne Android

Problème	Solution retenue
JDK système sans `javac` (JRE)	JDK Temurin 21 portable dans `tools/jdk/`, `JAVA_HOME` ancré dans les scripts
Gradle/AGP	Gradle 8.10.2 + AGP 8.7.3 + Chaquopy 17.2 (`python { version '3.13' }`, `buildPython` = venv du projet)
SDK Android	cmdline-tools + platform-34 + build-tools 34, installés dans `android-sdk/`
Packaging AGP incrémental (zipflinger) qui accumule les anciennes copies de `app.imy` (APK de 193 à 303 Mo)	`build.sh` supprime `build/outputs|intermediates/apk` avant chaque packaging

4.4. Phase D — Cross-build des wheels natifs

Les paquets Rust/C absents du dépôt de wheels Android sont **cross-buildés** vers `aarch64-linux-android` avec la recette suivante (`scripts/build-wheels.sh`) :

1. Python Android précompilé (artefact Maven `com.chaquo.python:python`) fournit `libpython3.13.so` et les en-têtes (`PYO3_CROSS_LIB_DIR`) ;
2. le NDK officiel ne fournissant que des binaires **x86_64**, le compilateur est le **clang hôte** (aarch64) pointé sur le **sysroot du NDK** (wrappers `tools/ndk-cc/`) ;
3. `libunwind.a` copié dans `tools/ndk-cc/lib/` (absent du NDK) pour le link ;

4. suffixe des modules : `-gnu.so` renommé `-android.so` (sinon l'import Android ne trouve pas le module) ; version des wheels corrigée quand setuptools-scm produit `0.0.0` (`scripts/fix-wheel-version.py`).

Wheel	Build	Raison de sa présence
`pydantic_core`	maturin	requis par pydantic v2 (cœur d'Open WebUI)
`jiter`	maturin	requis par le SDK `openai`
`uuid_utils`	maturin	requis par `langchain-core`
`orjson`	maturin	requis par `langsmith` (JSON rapide)
`rpds_py`	maturin	requis par `jsonschema` (via `mcp`)
`msgspec`	setuptools	requis par SearXNG (`_core.c` compilé avec les en-têtes du Python cible)
`typst`	maturin	compilateur Typst embarqué (outil `compile_typst`)

4.5. Phase E — Outils Android x86_64

Les outils officiels `aapt2` et `zipalign` sont des ELF **x86-64**. Ils tournent via **qemu-user-static** (Debian 12) avec un mini-sysroot amd64 (`libc6` + `libgcc-s1` , liens `lib64`) : wrappers `tools/wrappers/` , branchés par la propriété Gradle `android.aapt2FromMavenOverride` et le remplacement de `zipalign` dans les build-tools (original conservé en `zipalign.bin`).

4.6. Phase F — Pièges spécifiques

Piège	Correctif
`primp` (Rust) refuse le cross-build en API 24 : `getentropy` n'existe qu'à partir d'API 28	abandon de `ddgs` / `primp` — remplacé par SearXNG embarqué
Stt local `faster-whisper` (ctranslate2) inembarquable	STT par API ou **dictée native** (shim `SpeechRecognition` ⟶ `SpeechRecognizer`)
« no speech » : le `MediaRecorder` de l'UI monopolisait le micro	flux `getUserMedia` remplacé par un flux silencieux en mode STT « web » + relances automatiques de l'écoute
UI périmée après mise à jour (cache WebView)	purge `clearCache(true)` à chaque changement de `versionCode` (D13)
APK parle de « pas d'internet » alors que la recherche fonctionne	cause : ancien serveur de test resté vivant ; message d'erreur enrichi avec le moteur lu (P9b)
Commandes `fuser` / `pkill` inefficaces sous proot (tuent	toujours tuer par **PID** ; à ne jamais réutiliser dans les scripts

leur propre shell ou ne trouvent rien)	
Lecture SQLite `mode=ro` faussée par le WAL	toujours ouvrir en écriture pour lire l'état à jour

5. Ajouts par rapport à Open WebUI amont

Ajout	Composants	Détail
Base vectorielle locale `LocalVectorDB`	`src/open_webui/retrieval/vector/dbs/local.py`	SQLite + numpy, force brute vectorisée, score cosinus dans [0,1], filtres [$eq]/[$in] — `VECTOR_DB=local` par défaut
Provider **embedded-searxng**	P9/P9b/P10 + `src/searx/`	instance SearXNG locale port 8888, réglages générés avec clé secrète propre à l'appareil
Dictée vocale native	`AndroidSpeech.java` + shim JS	API `SpeechRecognition` du web ⟶ `SpeechRecognizer` Android (souvent hors-ligne) ; relances anti « no speech » ; arbitrage du micro
Outil **compile_typst**	P11 + wheel `typst`	compile un document `.typ` en PDF, l'enregistre dans les fichiers de l'utilisateur, affichable via `display_file`
Purge cache WebView	`MainActivity`	à chaque `versionCode` (D13), `version_code` incrémenté par `build.sh`
Écran de démarrage avec **barre de progression**	`MainActivity` + `android_main.py`	pourcentage réel (extraction des assets comptée fichier par fichier, étapes Python via `data/startup-status.txt` au format `PCT|message`), commentaires d'étape, compteur de secondes, et **messages d'erreur en rouge** en cas d'échec (import Python fautif, extraction…) — un blocage est visible et diagnostiquable
Foreground service	`ServerService.java`	le serveur survit au passage en arrière-plan
Fichier de configuration `data/openwebui.env`	`android_main.py`	variables `KEY=VALUE` chargées au démarrage, sans rebuild
Notification de permission contextuelle	`MainActivity`	demande micro/caméra à l'appel `getUserMedia`, pas seulement au lancement

6. Patchs du code amont

Tous les patchs sont générés par `scripts/prepare_src.py` (idempotent). Le document de référence est `PATCHES.md` ; résumé opérationnel :

N°	Cible	Rôle
P1	`config.py`	`VECTOR_DB=local` par défaut
P2	`retrieval/vector/type.py`	types `NONE` et `LOCAL`
P3	`retrieval/vector/factory.py`	aiguillage `LOCAL` / `NONE` vers les backends embarqués
P4	`tools/knowledge_fs.py`	repli `re2` ⟶ `re`
P5	`routers/audio/__init__.py`	`pydub` optionnel (`audioop` absent de la stdlib 3.13)
P6	`storage/provider.py`	SDK Azure/GCS optionnels (protobuf/grpcio absents)
P7a-c	`routers/audio/__init__.py`	STT/TTS locaux ⟶ erreur 503 explicite
P8	`config.py`	moteur STT par défaut « web » (dictée native)
P9/P9b	`routers/retrieval.py`	provider `embedded-searxng` + message d'erreur portant le moteur lu
P10	`frontend/.../nodes/*.js`	ajout de `embedded-searxng` à la liste des providers
P11	`tools/builtin.py` + `utils/tools.py`	outil `compile_typst` et son enregistrement
P12	`utils/tools.py`	`features.get('web_search', True)` : les outils internet (`search_web` = `embedded-searxng` , `fetch_url`) sont exposés même quand le modèle ne déclare pas la capacité dans ses métadonnées — **supersédé par P18**
P18	`utils/tools.py`	gate des outils web réduit à `web.search.enable` + permission : `search_web` / `fetch_url` exposés quelles que soient les capacités modèle (`capabilities.web_search` , `builtinTools.web_search`) et le toggle `features` de la requête (souvent `false` explicite pour les API tierces ou globe non actionné)
P13	`frontend/` (données)	allègement : suppression des **source maps**, de `pyodide/` et `wasm/` (169 Mo) — ML navigateur et Python-navigateur désactivés,

		conformes à la règle « pas de calcul local » ; gain majeur de démarrage et de taille APK
P14	`routers/auths.py`, `routers/utils.py`, `routers/configs.py`	imports différés : `ldap3` (1,7 s), `black` (0,9 s), `MCPClient` / `mcp` (1-2 s) n'exécutent plus au démarrage — importés à l'usage
P15	`src/` ⟶ `src-assets/`	le frontend sort des **assets Python** (extrait à l'import par Chaquopy) pour devenir des **assets Android**, extraits une fois par version par `MainActivity` avec progression ; `app.imy` et l'import Python s'effondrent
P16	`open_webui/migrations/`	`migrations/` et `versions/` marqués paquets + `extractPackages` : les `.py` restent sur disque malgré la compilation bytecode — Alembic **lit** `env.py` / `versions/*.py` comme des fichiers (correctif du blocage « … migrations/env.py » sur l'écran de démarrage)

6.1. Performances de démarrage

Le démarrage combine : extraction des données Python par Chaquopy, l'import d'Open WebUI, puis le lifespan (migrations, init). Trois optimisations successives, mesurées sur le CPU de l'appareil (temps jusqu'à `/health`) :

Étape	Avant	Après
Import du serveur (mesuré)	35,5 s	≈ 11 s
Taille du paquet Python `src/open_webui`	264 Mo	9,9 Mo
Taille APK	245 Mo	≈ 150 Mo

- P13 : frontend allégé (248 ⟶ 85 Mo : source maps, `pyodide/`, `wasm/`) ;
- P14 : imports coûteux différés (`ldap3`, `black`, `MCPClient`) ;
- P15 : le frontend est extrait **par Java, une fois par version** (avec message « Préparation de l'interface »), plus par l'import Python à chaque lancement ;
- SearXNG démarre **après** `/health` (pas de contention sur l'écran d'attente) ;
- l'écran de démarrage affiche l'étape réelle + un compteur de secondes (`data/startup-status.txt`) ; les **erreurs d'import du serveur y sont écrites** — un blocage devient donc lisible, pas silencieux.

Si un démarrage redevient long : lire l'écran (étape + compteur) ; si le texte mentionne une erreur, c'est l'import Python fautive (manque de dépendance) ; sinon, vérifier `adb logcat | grep python`.

Bonne pratique

> Si un patch échoue (`PATCH EN ÉCHEC : motif introuvable`), c'est que le code amont a changé à cet endroit : ouvrir le fichier concerné dans `vendor/unpacked/` , adapter le **motif** dans `prepare_src.py` , recompiler. Ne jamais éditer `src/` à la main : il est régénéré à chaque appel.

7. Mise à jour d'Open WebUI

1.	Récupérer la nouvelle version : `tools/python312/bin/python3.12 -m pip download open-webui --no-deps -d vendor/wheel` puis décompresser le wheel dans `vendor/unpacked/` (écraser l'ancien **pristine**).
2.	Examiner le nouveau `requirements-slim.txt` et le `METADATA` : nouvelles dépendances ? natives ? versions Python ?
3.	Rejouer `scripts/prepare_src.py` : chaque patch qui échoue signale un changement amont à intégrer à la main. Mettre à jour `PATCHES.md`.
4.	Adapter `requirements-min.txt` : tout nouveau paquet **natif** doit être (a) présent dans le dépôt de wheels Chaquopy (`https://chaquo.com/pypi-13.1/`), ou (b) cross-buildé via `build-wheels.sh`, ou (c) remplacé par un repli documenté. Vérifier les plafonds connus : `numpy≤1.26.2`, `cryptography≤42.0.8`, `bcrypt≤3.2.2`, `argon2-cffi` = 23.1.0, `authlib≤1.6.4`, `joserfc≤1.5.0`.
5.	Test de fumée hôte : `scripts/smoke.sh` puis vérifier `/health`, la connexion, la recherche web et SearXNG (`/search?q=test&format=json`).
6.	Build : `bash scripts/build.sh` ⟶ `dist/app-debug.apk`, contrôle de taille (écart zip < 10 Ko) et du `badging`.
7.	Test appareil (chat, dictée, recherche, upload document avec embeddings, Typst) puis mise à jour de `journal.md`, `PATCHES.md`, `todo.md`, et du `versionName`.

> Ne jamais sauter l'étape 5 : la majorité des régressions (imports supprimés, signatures changées, nouvelles dépendances natives) se voient au démarrage du serveur bien avant le build Android — et coûtent 45 minutes de build en moins.

8. Mise à jour des composants

Composant	Emplacement	Procédure
Wheels Rust/C	`wheels/` via `scripts/build-wheels.sh`	ajouter/modifier la version dans le bloc `for spec` (sdists dans `downloads/sdists/`), relancer le script. `pydantic_core` suit la version exigée par `pydantic` ; `jiter` suit `openai` ; `msgspec` suit SearXNG.
SearXNG	`vendor/searxng/`	re-cloner puis `git checkout <nouveau-commit>` — **vérifier avant** que `requirements.txt` ne contient ni `curl_cffi`, ni `fasttext`, ni `uvloop`, ni version de `msgspec` incompatible ; rejouer `prepare_src.py`.
Chaquopy	`build.gradle` (plugin) + `tools/python312`	mettre à jour le plugin et le `python { version }` ; `buildPython` doit avoir le même **major.minor** que la cible.
AGP / Gradle	`build.gradle` / `scripts/setup-sdk.sh`	paire AGP-Gradle compatible ; redécompresser Gradle dans `tools/gradle/`.
SDK Android / NDK	`android-sdk/`, `tools/android-ndk/`	relancer `setup-sdk.sh` (platform/build-tools) ; le NDK ne sert qu'au **sysroot** pour le cross-build Rust.
qemu + sysroot amd64	`tools/qemu/`	à changer seulement si les outils `aapt2` / `zipalign` changent de glibc ; le sysroot vient des `.deb` Debian (`libc6`, `libgcc-s1`).
JDK	`tools/jdk/`	remplacer le contenu ; `JAVA_HOME` est ancré dans `env.sh` / `build.sh`.
Open WebUI	`vendor/unpacked/` + `src/`	procédure dédiée : section précédente.

9. Construire l'APK

```
source scripts/env.sh            # ancrage d'isolation (INDISPENSABLE)
bash scripts/build.sh            # → dist/app-debug.apk (debug)
bash scripts/build.sh assembleRelease   # variante release (keystore requis)
```

Le `versionCode` est incrémenté automatiquement (`fichier version_code`) : c'est lui qui déclenche la purge du cache WebView au premier lancement de la nouvelle version. Un build propre ne doit jamais laisser l'APK grossir de plus d'une dizaine de Mo sans explication : l'écart `taille du fichier - somme des entrées zip` doit rester de l'ordre de 8 Ko.

10. Configuration sur l'appareil

Chemin (stockage privé de l'app)	Contenu
`files/data/webui.db`	base applicative (utilisateurs, chats, réglages)
`files/data/uploads/`	fichiers téléversés, nommés `{uuid}_{nom d'origine}`
`files/data/vector_db/local_vectors.sqlite3`	vecteurs RAG locaux
`files/data/searxng-settings.yml`	réglages SearXNG (clé secrète générée)
`files/data/openwebui.env`	variables d'environnement optionnelles (`KEY=VALUE`)
`files/tmp/`	temporaires Python (`TMPDIR`)
`files/cache/`	caches diverses (`XDG_CACHE_HOME`)

Depuis v1.0.1, tous ces chemins découlent explicitement de `getFilesDir()` transmis par `MainActivity` à `start_server` — `HOME` , `TMPDIR` et `XDG_CACHE_HOME` sont **forcés** dans l'espace privé (`/data/data/org.openwebui.android/files/…`) : aucun fichier n'est écrit ailleurs.

Réglages essentiels depuis l'UI : **Admin → Documents** (moteur d'embedding distant
1. clé), **Admin → Audio** (STT « web » pour la dictée), **Admin → Recherche web**

(moteur `embedded-searxng`). Depuis v15, `android_main` force `web.search.enable` à `true` au démarrage (`ENABLE_WEB_SEARCH`) ; depuis v16 (P18), les outils `search_web` / `fetch_url` sont exposés **sans condition de modèle ni de toggle** — y compris pour les API tierces (DeepSeek…) dont la fiche modèle porte `capabilities.web_search: false` , et quand le globe du chat n'est pas actionné. Le pipeline de recherche automatique (globe) reste, lui, conditionné au toggle.

11. Dépannage rapide

Symptôme	Cause probable	Correctif
L'UI affiche une ancienne version	cache WebView	réinstaller + forcer l'arrêt / vider le cache ; normalement automatique via le `versionCode`
« no speech » à la dictée	micro occupé ou permissions	accorder micro/caméra ; l'app relance déjà l'écoute 3 fois
Recherche web vide	moteurs amont bloqués (IP datacenter) ou moteur mal configuré	vérifier `Admin → Recherche web` ; le message d'erreur porte désormais le moteur lu
Pas d'outils web (`search_web` / `fetch_url` absents de l'inventaire)	`web.search.enable` éteint (`ENABLE_WEB_SEARCH` défaut `False` en amont), ou capacités modèle / toggle `features` à `false` explicite	v15 force `web.search.enable` ; v16 (P18) ignore capacités et toggle — si absent malgré tout : `sqlite3 webui.db \"SELECT value FROM config WHERE key='web.search.enable'\"`
RAG inactif au téléversement	pas d'API d'embeddings configurée	configurer **Admin → Documents** (embeddings distants obligatoires)
Build pip : `from versions: none`	paquet natif absent du dépôt Android	cross-build (maturin/setuptools) ou retirer la dépendance — cf. Chapitre 8
Build pip : conflit `numpy` / `grpcio-tools`	dépendance exigeant des versions natives supérieures	pinner une version amont antérieure compatible
L'APK grossit anormalement	packaging incrémental	`build.sh` nettoie déjà ; vérifier qu'on ne passe pas par un autre script

« Erreur au démarrage : … migrations/env.py »	Chaquopy retire les `.py` après compilation bytecode, or Alembic lit les fichiers de `migrations/`	patch P16 (`extractPackages`) — vérifier la présence de `open_webui/migrations/__init__.py` dans `src/`
CI : `pip download open-webui` refuse 0.13.0 « from versions: none »	le wheel amont porte `Requires-Python <3.13` (cap pydub/audioop, cf. D7) et la CI télécharge en 3.13	`fetch_src.sh` passe `--ignore-requires-python` (wheel `py3-none-any` — le cap ne s'applique pas)
« dlopen failed … .abi3.so » (uuid_utils, typst)	les wheels **abi3** (PyO3) ne lient pas `libpython` : symboles `Py*` indéfinis au chargement — fatal sous Android	`build-wheels.sh` force la liaison via `PYO3_CONFIG_FILE` (`suppress_build_scrip` vérifier par `readelf -d <module>.so | grep NEEDED` (doit mentionner `libpython3.13.so`), puis rebuild des wheels — plan B : `patchelf --add-needed libpython3.13.so`
Le correctif wheel « ne prend pas » (même erreur après rebuild)	la tâche pip de Chaquopy ne suit que le **fichier** de requirements : un wheel reconstruit (même nom/version) n'est pas réinstallé — l'ancien `.so` reste dans l'APK	`build.sh` vide `app/build/python| + generated intermediates/assets` à chaque build ; vérifier le `.so` de `app/build/python/pip/debug/common/` par `readelf` avant livraison
Le serveur ne démarre pas sur l'appareil	exception d'import	lire logcat (`adb logcat | grep python`) ; cause fréquente : nouvelle dépendance non embarquée

12. Règle d'isolation de l'environnement

Tout doit rester dans le répertoire du projet : SDK, Gradle, Rust, NDK, JDK, qemu, caches, venv. Aucune installation globale, aucune écriture dans le répertoire personnel de l'utilisateur. Les scripts exportent les variables (`ANDROID_HOME` , `GRADLE_USER_HOME` , `CARGO_HOME` , `RUSTUP_HOME` , `JAVA_HOME` , `XDG_CACHE_HOME` , `TMPDIR`) ; `scripts/env.sh` est la porte d'entrée unique.

> Leçon du projet : `rustup-init` ajoute une ligne dans `~/.profile` **même** avec `CARGO_HOME` déplacé — toujours passer par `--no-modify-path` . De même, un `cargo` lancé hors `env.sh` recrée `~/.cargo` . Toute commande manuelle Rust, pip ou Gradle doit être précédée de `source scripts/env.sh` .

> **Bonne pratique**
>
> Pour vérifier l'absence de pollution : `find ~ -maxdepth 1 -newermt <date>` en dehors du projet ; et relire `journal.md` qui trace les incidents passés.

13. Annexes

13.1. Versions verrouillées à la livraison initiale

Composant	Version
Open WebUI	0.13.0
Python cible (Chaquopy)	3.13
Chaquopy / AGP / Gradle	17.2.0 / 8.7.3 / 8.10.2
SearXNG	commit `a30b2d4749` (2026-08-28)
typst / msgspec	0.15.2 / 0.21.1
pydantic / pydantic-core	2.13.4 / 2.46.4
SDK Android / build-tools	platform 34 / 34.0.0
NDK (sysroot)	r27c

13.2. Commandes utiles

```
bash scripts/build.sh          # build complet
bash scripts/build-wheels.sh  # wheels Rust uniquement
bash scripts/prepare_src.py   # régénérer src/ (patchs)
bash scripts/smoke.sh         # test de fumée hôte
adb install -r dist/app-debug.apk
```

13.3. Règles de test hôte (serveur de développement)

Le test hôte (`scripts/smoke.sh`) vérifie le code embarqué sans appareil. Deux pièges invalident tout test s'ils ne sont pas contrôlés :

- *Serveurs zombies* : `fuser` / `pkill` sont inefficaces sous proot ; un ancien serveur peut rester sur le port 8080 et répondre avec du code périmé. Toujours tuer par PID : `pgrep -f 'venv313/bin/pyth[o]n'` (classe de caractères pour ne pas tuer la commande de kill elle-même), puis vérifier `pgrep -f "$PAT" | wc -l` vaut 0.
- *Démarrage lent* : le serveur met environ 40 s à être prêt. Attendre `/health` en boucle (requête toutes les 3 s) avant tout test — un `sleep` court produit des faux négatifs.

Un seul serveur par test ; ne jamais inférer un bug applicatif d'un test dont l'identité du serveur n'est pas confirmée.

13.4. Intégration continue (GitHub Actions)

`.github/workflows/build-apk.yml` build les APK *debug* et *release* uniquement au push d'un tag `v*`, et publie la Release GitHub avec les APK attachés. Adaptations automatiques du runner x86_64 : retrait de la surcharge `aapt2FromMavenOverride` (qemu), JDK système accepté si `tools/jdk` (aarch64) est absent, `.venv313` recréé en CPython 3.13, SearXNG cloné au commit figé. Le `versionCode` CI vaut `10000 + numéro d'exécution`. Signature release : secrets GitHub `KEYSTORE_BASE64` , `KEYSTORE_PASSWORD` , `KEY_ALIAS` , `KEY_PASSWORD` — sans eux, l'APK release est signé avec la clé debug (installable, pour tests).

13.5. Pistes d'évolution

- Mise à jour in-app de l'APK (GitHub Releases) ;
- profils multiples de configuration (`openwebui.env` prêt) ;
- base vectorielle locale : passage à un index ANN (hnswlib) si les volumes dépassent 100 000 fragments ;
- signature release et publication (Play Store non visé).